

UML COMPLETE EXAM GUIDE

Use Case • Class Diagram • Sequence Diagram • StarUML • Viva Prep

- One-Night Crash Course for Software Modeling & Architecture
 - Beginner-Friendly | Exam-Focused | StarUML Ready
- Every Symbol • Every Arrow • Every Relationship Explained
- With ASCII Diagrams • Viva Q&A; • Common Mistakes

SECTION 1 — USE CASE DIAGRAM

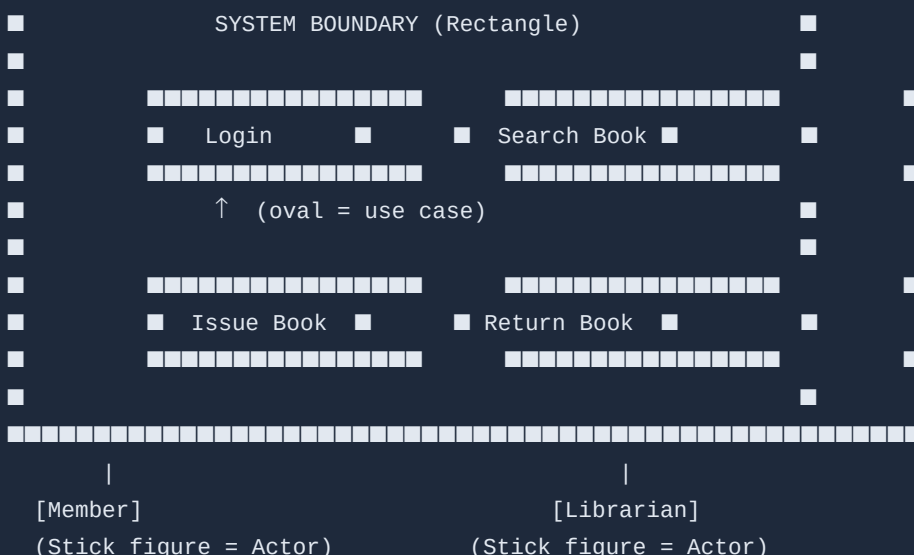
A Use Case Diagram shows WHO interacts with the system and WHAT they can do. It captures functional requirements from the user's perspective. Think of it as a 'bird's eye view' of the system's capabilities.

1.1 What Is a Use Case Diagram?

A Use Case Diagram answers three questions:

1. WHO uses the system? → Actors
2. WHAT can they do? → Use Cases (ovals)
3. HOW are they connected? → Relationships (lines & arrows)

■ ASCII DIAGRAM

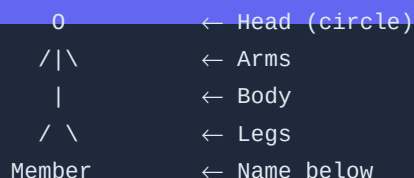


1.2 The 7 Core Elements — Complete Visual Guide

■ ACTOR

Definition: A person, organization, or external system that interacts with YOUR system.

■ ASCII DIAGRAM

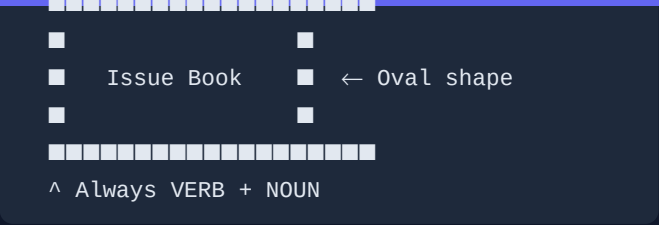


- Symbol: Stick figure (outside system boundary)
- Rule: Actor is ALWAYS outside the rectangle boundary
- Types: Primary (initiates action) vs Secondary (reacts/supports)
- Example: Customer, Librarian, Admin, Bank System, Payment Gateway

■ USE CASE

Definition: A specific function or service the system provides to an actor. It represents a goal the actor wants to achieve.

■ ASCII DIAGRAM

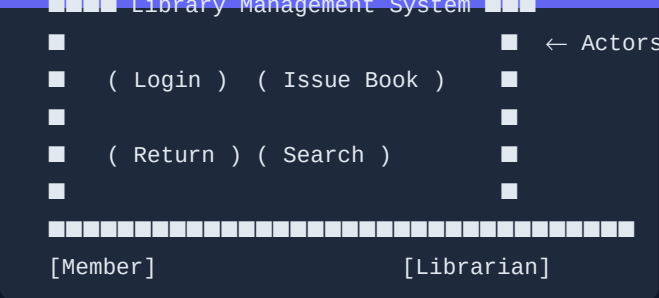


- Symbol: Oval / Ellipse (inside system boundary)
- Naming: Use VERB + NOUN format (e.g., 'Issue Book', 'Pay Bill')
- Rule: Always inside the system boundary rectangle
- Tip: Each use case = one complete user goal

■ SYSTEM BOUNDARY

Definition: A rectangle that defines what is INSIDE your system vs what is OUTSIDE.

■ ASCII DIAGRAM

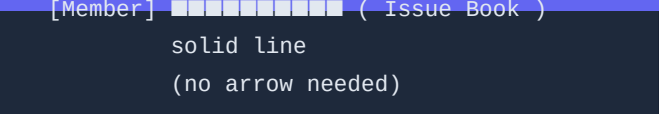


- Symbol: Rectangle with system name at top
- Actors go OUTSIDE the rectangle
- Use Cases go INSIDE the rectangle
- Shows the scope of the system being designed

■ ASSOCIATION

Definition: A simple solid line connecting an Actor to a Use Case, showing the actor participates in that use case.

■ ASCII DIAGRAM



- Actor to Use Case = OK
- Use Case to Use Case = NOT association

- Symbol: Plain solid line (no arrowhead needed)

- Direction: Actor — Use Case
- Meaning: 'This actor uses this functionality'
- Rule: Only between Actor ↔ Use Case, NOT between two use cases

■ INCLUDE

Definition: Shows that one use case ALWAYS and MANDATORILY calls another use case. The included use case is required every single time.

■ ASCII DIAGRAM

```
( Issue Book ) -----> ( Verify Member )
                <<include>>
                ↑           ↑
        Base use case   Included (mandatory)
```

Arrow points FROM base TO included
Dashed line + <<include>> label

- Symbol: Dashed arrow with <> label
- Direction: Base → Included (arrow points TO the included case)
- Meaning: 'Cannot happen without this'
- Keyword: ALWAYS, MANDATORY, EVERY TIME
- Memory trick: <> = 'must include' = mandatory

■ EXTEND

Definition: Shows that one use case SOMETIMES adds optional behaviour to another use case. It only happens under certain conditions.

■ ASCII DIAGRAM

```
( Calculate Fine ) -----> ( Return Book )
                <<extend>>
                ↑           ↑
        Extension (optional)   Base use case
```

Arrow points FROM extension TO base
Only happens IF book is overdue

- Symbol: Dashed arrow with <> label
- Direction: Extension → Base (arrow points TO the base case)
- Meaning: 'This sometimes happens'
- Keyword: SOMETIMES, OPTIONAL, IF condition
- Memory trick: <> = 'extra optional feature'

■ GENERALIZATION

Definition: Inheritance relationship. A child actor/use case inherits all properties of parent and can add its own specific behaviour.

■ ASCII DIAGRAM

```
[Admin] ■■■■■■ [User]
```



hollow triangle arrowhead
points TO parent (User)

Admin IS-A User (inherits all User capabilities)
Admin also has extra admin-only capabilities

- Symbol: Solid line with HOLLOW arrowhead (triangle)
- Direction: Child → Parent (arrow points to parent)
- Use for Actors: When one actor is a specialization of another
- Use for Use Cases: When one use case is a type of another
- Memory trick: Arrow points UP to parent (like family tree)

1.3

INCLUDE vs EXTEND — The Most Important Comparison

Feature	<<include>>	<<extend>>
Meaning	Always happens	Sometimes happens
Type	Mandatory	Optional/Conditional
Arrow direction	Base → Included	Extension → Base
When to use	Common sub-step needed by multiple use cases	Optional feature that happens under a condition
Keyword trigger	always, must, every time	sometimes, if, optionally
Example	Book Ticket includes Verify Payment	Cancel Ticket extends with Refund (if eligible)
Memory trick	I INCLUDE you always	I EXTEND only sometimes

■ ASCII DIAGRAM

INCLUDE Example:

```
( Book Ticket ) ----<<include>>----> ( Verify Payment )
^Base use case      arrow→      ^Included (mandatory)
```

EXTEND Example:

```
( Send SMS Alert ) ----<<extend>>----> ( Book Ticket )
^Extension (optional)  arrow→      ^Base use case
```

KEY RULE: Arrow ALWAYS points TO the use case being 'used'

include: arrow → included case (which is called/used)

extend: arrow → base case (which is being extended/modified)

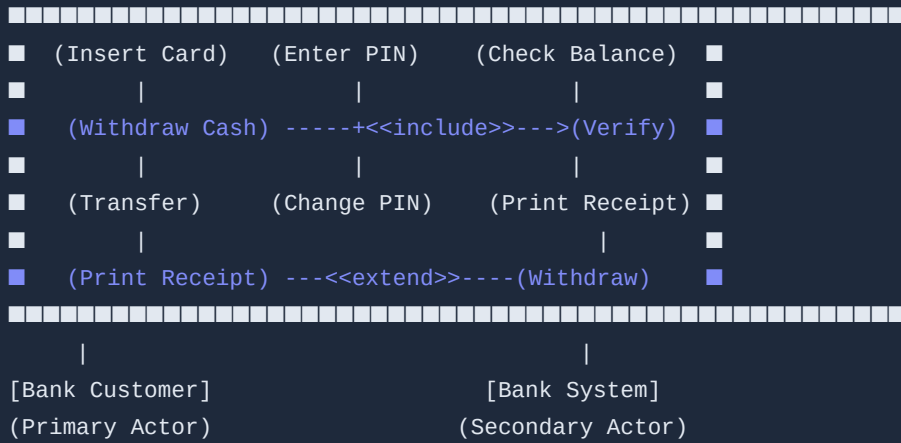
1.4

Complete Use Case Diagrams with Full Explanation

ATM System

■ ASCII DIAGRAM

ATM SYSTEM

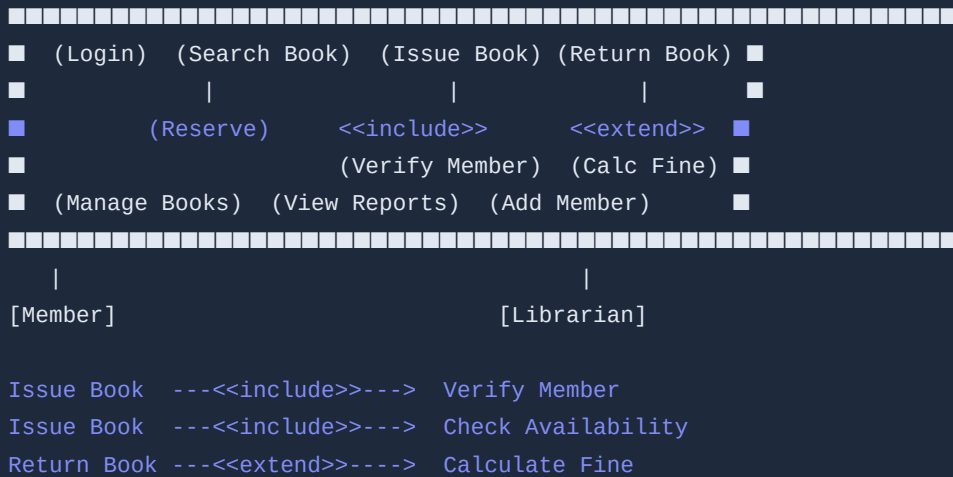


Element	Type	Reason
Bank Customer	Primary Actor	Initiates all transactions
Bank System	Secondary Actor	Validates, processes transactions
Insert Card	Use Case	First step to use ATM
Verify PIN	Use Case (included)	Withdraw includes Verify PIN
Withdraw <<include>> Verify	Include	Can't withdraw without PIN verification
Print Receipt <<extend>> Withdraw	Extend	Receipt is optional after withdrawal

Library Management System

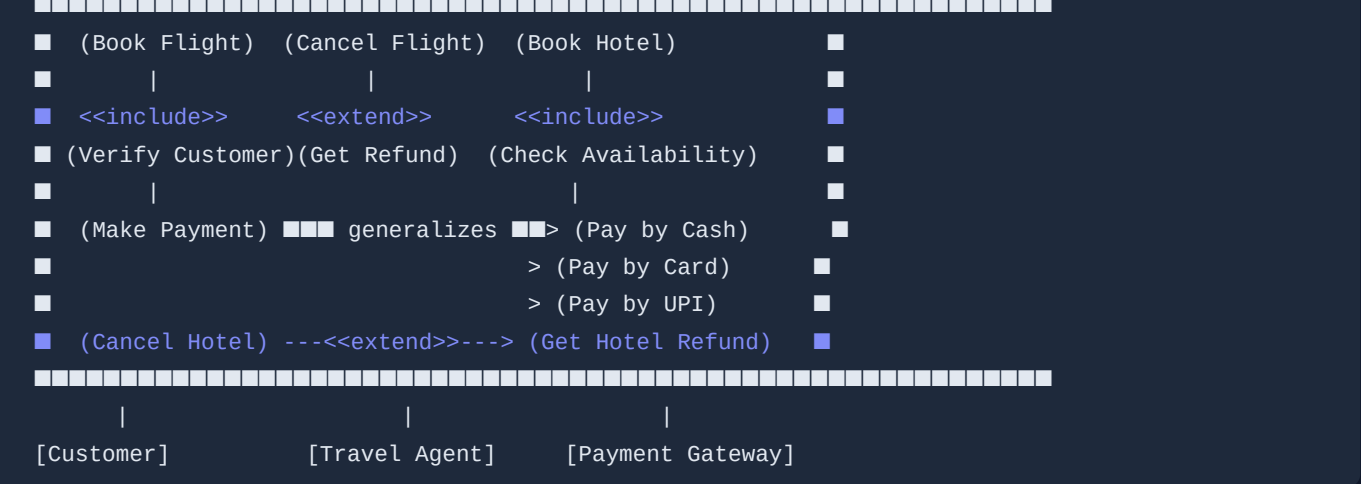
■ ASCII DIAGRAM

LIBRARY MANAGEMENT SYSTEM



Relationship	Why Used
Issue Book <<include>> Verify Member	Every book issue MUST verify member — mandatory
Issue Book <<include>> Check Availability	Book must be available before issuing — mandatory
Return Book <<extend>> Calculate Fine	Fine only if overdue — conditional/optional
Member → Login	Member must log in to access system

TRAVEL AGENCY SYSTEM



The generalization of 'Make Payment' into 'Pay by Cash', 'Pay by Card', and 'Pay by UPI' shows that all three ARE-A type of payment — they inherit the payment behaviour but each has a different implementation.

SECTION 2 — CLASS DIAGRAM

A Class Diagram shows the STATIC STRUCTURE of a system — what classes exist, what data they hold (attributes), what they can do (methods), and how they relate to each other. It is the blueprint of your code.

2.1 Anatomy of a Class

■ ASCII DIAGRAM

```

■          ClassName          ■ ← Section 1: Class Name (Bold, Centred)
■
■ - attribute1 : DataType    ■ ← Section 2: Attributes (data/fields)
■ - attribute2 : DataType    ■
■ + publicAttr : String      ■
■ # protectedAttr : int      ■
■
■ + method1() : ReturnType    ■ ← Section 3: Methods (operations)
■ - privateMethod() : void    ■
■ + method2(param : int) : bool■

```

VISIBILITY SYMBOLS:

+ = public (anyone can access)
- = private (only THIS class)
= protected (this class + subclasses)
~ = package (classes in same package)

Symbol	Name	Who Can Access	When to Use
+	Public	Any class anywhere	Methods users of class need to call
-	Private	Only the class itself	Internal data, implementation details
#	Protected	Class + all subclasses	Data shared with child classes
~	Package	Classes in same package	Package-internal helpers

Real Class Example: BankAccount

[illegible]

2.2 Class Diagram Relationships — Complete Guide

Symbol: Solid line, no special arrowhead (or open arrow for directed)

```
Customer ██████████ Order
      1             0..*
'A Customer places Orders'
Both classes exist independently
Customer knows about Order
```

Symbol: Dashed line with open arrowhead (--->)

```
PaymentProcessor - - -> BankAPI
(dashed arrow)
'PaymentProcessor temporarily uses BankAPI'
PaymentProcessor does NOT store BankAPI permanently
It just calls a method on it and is done
```

Page 10 | UML Complete Exam Guide

Symbol: Solid line with HOLLOW diamond at the WHOLE end

■ ASCII DIAGRAM

```
Library —◆ Book
(hollow diamond at Library end)
Library HAS books
But books CAN EXIST without the library
(books don't die when library closes)
```

When to Use: Use when parts can exist independently. The 'whole' just aggregates the parts temporarily.

■ COMPOSITION

Definition: 'Part-of' strong relationship. The PART CANNOT exist without the WHOLE. Like a house and its rooms.

Symbol: Solid line with FILLED (solid) diamond at the WHOLE end

■ ASCII DIAGRAM

```
Order ◆ OrderItem
(filled diamond at Order end)
Order OWNS OrderItems
If Order is deleted, OrderItems die too
OrderItem CANNOT exist without Order
```

When to Use: Use when part cannot exist independently. Destruction of whole destroys the parts.

■ INHERITANCE / GENERALIZATION

Definition: 'Is-a' relationship. Child class inherits all attributes and methods of parent, can override and extend.

Symbol: Solid line with HOLLOW TRIANGLE arrowhead pointing to PARENT

■ ASCII DIAGRAM

```
SavingsAccount —| BankAccount
CurrentAccount —| BankAccount
(hollow triangle points TO parent)
SavingsAccount IS-A BankAccount
It inherits all BankAccount features
+ adds its own (interest calculation)
```

When to Use: Use for IS-A relationships. Child is a specialized version of parent.

■ REALIZATION / INTERFACE

Definition: Class implements an interface — it promises to provide all the methods declared in the interface.

Symbol: Dashed line with HOLLOW TRIANGLE arrowhead pointing to INTERFACE

■ ASCII DIAGRAM

```
CreditCardPayment -.-| <<Payable>>
UPIPayment -.-| <<Payable>>
(dashed line + hollow triangle)
Both classes IMPLEMENT the Payable interface
Both MUST provide processPayment() and refund()
```

When to Use: Use when a class promises to implement all methods of an interface (contract).

■ ASCII DIAGRAM

```

■ Customer ■ 1 0..* ■ Order ■
■-customerId ■ ■-orderId ■
■-name ■ ■-status ■
■-email ■ ■-date ■
■-totalAmount ■
■+login() ■
■+viewOrders() ■ ■+placeOrder() ■
■+cancelOrder()■
■ ♦ 1
1..* ■
■ Product ■ ■ OrderItem ■
■-productId ■ ■-quantity ■
■-name ■ ■-unitPrice ■
■-price ■
■-stock ■
■+getSubtotal()■
■+getDetails() ■
■+checkStock() ■
■ implements
■ - - - - ■
■ <<Purchasable>>■
■+getPrice():dbl ■
■+isAvailable(): ■

```

Banking System

■ ASCII DIAGRAM



SavingsAccount and CurrentAccount both INHERIT from BankAccount (IS-A relationship). Customer COMPOSES BankAccount (account cannot exist without customer).

Library System Class Diagram

■ ASCII DIAGRAM

```

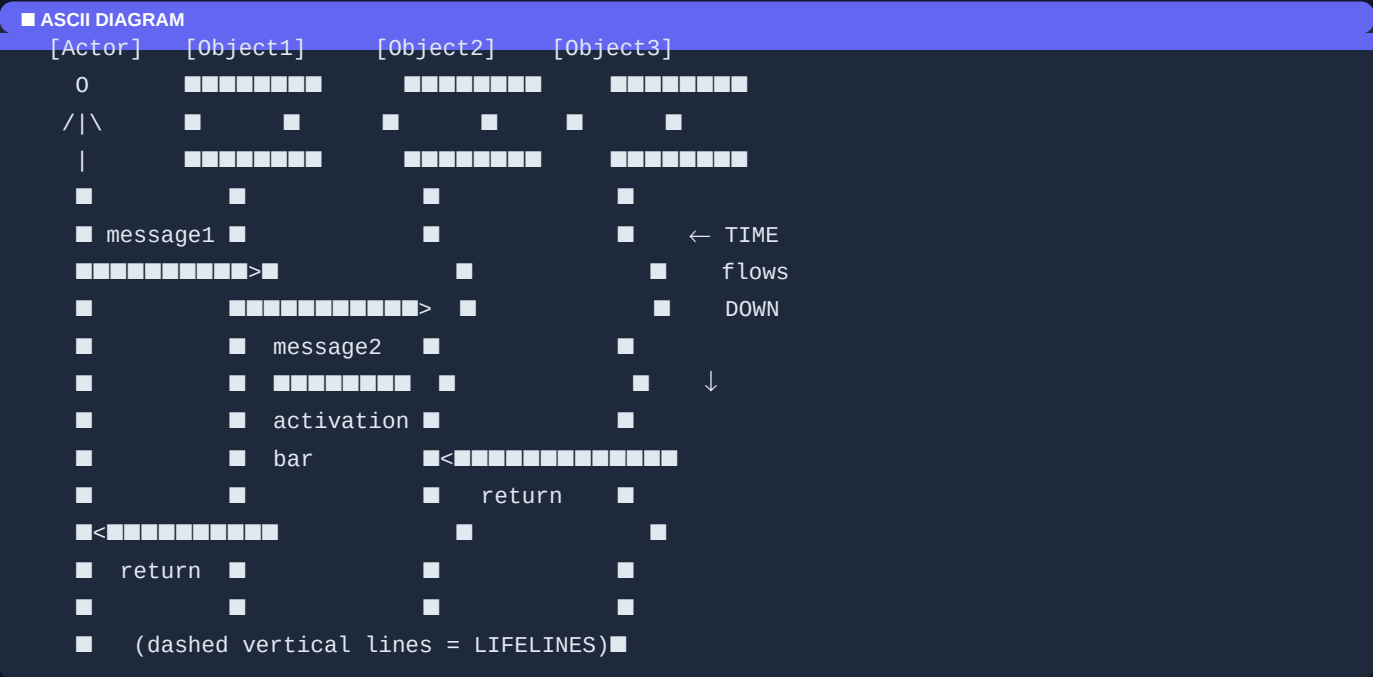
■ Member ■ 1 0..* ■ BorrowRecord■
■ ■ ■ ■ ■
■-memberId ■ ■-issueDate ■
■-name ■ ■-dueDate ■
■-email ■ ■-returnDate ■
■ ■ ■-fine: double ■
■+issueBook() ■ ■
■+returnBook()■ ■+calcFine() ■
■ ■ ■
■ ■ 1 1 ■
■ Book ■ ■ ■
■ ■ ■
■-ISBN ■ ■
■-title ■ ■
■-author ■ ■
■-copies ■ ■
■ ■ ■
■+getDetails()■ ■
■+isAvail() ■ ■
■ ■ ■
■ ■ ■
■ Librarian ■ manages
■ ■
■-employeeId ■
■ ■
■+addBook() ■
■+removeBook()■
■ ■

```

SECTION 3 — SEQUENCE DIAGRAM

A Sequence Diagram shows HOW objects interact in TIME ORDER. It captures the flow of messages between objects to complete a specific scenario (use case).

3.1 Core Elements of Sequence Diagram



- **Objects / Participants:** Boxes at the TOP of the diagram. Can be actors (stick figures) or objects (rectangles). Named as 'ObjectName : ClassName' (e.g., ':Customer' or 'cart:Cart')
- **Lifelines:** Dashed vertical lines extending DOWN from each object. Represents the object's existence over time.
- **Activation Bars:** Thin filled rectangles ON the lifeline. Shows when an object is ACTIVELY processing something. Appears during method execution.
- **Synchronous Message** →: Solid arrow. Sender WAITS for receiver to complete and return. Most common type. The sender is blocked until response.
- **Return Message** ←: Dashed arrow going BACK. Shows the return value or response from a method call.
- **Asynchronous Message** →>: Solid arrow with half-arrowhead. Sender does NOT wait — fires and continues. Used for events, notifications.
- **Self-Call:** Arrow that loops back to SAME object. Object calls its own method internally.
- **Loop Fragment:** Rectangle labeled 'loop' around a set of messages. Repeats a sequence.
- **Alt Fragment:** Rectangle labeled 'alt' with dividing line. Shows IF-ELSE conditional flow.
- **Opt Fragment:** Rectangle labeled 'opt'. Optional sequence — only happens if condition is true.

3.2

Complete Sequence Diagrams with Full Explanation

ATM Withdrawal

■ ASCII DIAGRAM

```

[Customer]  [ :ATMScreen]  [ :AccountSystem]  [ :Bank]
0           ██████████      ██████████      ██████████
/|\        █      █      █      █
|          ██████████      ██████████      ██████████
█          █              █              █
█insertCard>█              █              █
█          █              █              █
█<██████████>█ promptPIN()  █              █
█          █              █              █
█enterPIN█>█              █              █
█          █████verifyPIN()████>█              █
█          █              █              █
█          █              █████checkPin()█>█
█          █              █              █
█          █              █<█pinValid█████
█          █<████pinOK█████████              █
█<█showMenu█              █              █
█          █              █              █
█selectWithdraw>█              █              █
█enterAmount█>█              █              █
█          █████processWithdraw()████████████████>█
█          █              █              █
█          █              █<█deductAmt█████
█          █<████success████████              █
█<████dispenseCash█              █              █
█<█printReceipt█              █              █

```

Message	From → To	Why
insertCard	Customer → ATMScreen	Starts the interaction
verifyPIN	ATMScreen → AccountSystem	ATM delegates verification to backend
checkPin	AccountSystem → Bank	Bank has the actual PIN database
pinValid	Bank → AccountSystem	Return value from bank verification
processWithdraw	ATMScreen → Bank	Final transaction at the bank
dispenseCash	ATMScreen → Customer	Physical/UI response to customer

Online Shopping Checkout

■ ASCII DIAGRAM

```

[User] -- [Browser] -- [WebService] -- [OrderService] -- [PaymentCU]

```

[illegible]

■ ASCII DIAGRAM

```

[User]      [LoginPage]  [AuthService]  [UserDB]
0          ■■■■■■■■■■    ■■■■■■■■■■    ■■■■■■■■■■
/|\        ■          ■      ■          ■      ■          ■
|          ■■■■■■■■■■    ■■■■■■■■■■    ■■■■■■■■■■
■          ■              ■              ■              ■
■■openApp■>■              ■              ■              ■
■<■showLogin■            ■              ■              ■
■          ■              ■              ■              ■
■■enterCreds■>■          ■              ■              ■
■          ■■■authenticate()>■          ■              ■
■          ■              ■              ■              ■
■          ■              ■■■findUser()■>■          ■
■          ■              ■              ■              ■
■          ■              ■<■■userData■■■■■          ■
■          ■              ■              ■              ■
■ alt [credentials valid]          ■              ■
■ ■          ■<■■authToken■■■          ■              ■
■ ■<■redirect to Dashboard          ■              ■
■ else [invalid]          ■              ■
■ ■<■showError('Invalid login')          ■              ■
■ end alt          ■              ■

```

■ ASCII DIAGRAM

[Customer] [:BookingUI] [:FlightSvc] [:PaymentSvc] [:EmailSvc]

```

0      ■■■■■■■■      ■■■■■■■■      ■■■■■■■■      ■■■■■■■■
/|\    ■      ■      ■      ■      ■      ■      ■
|      ■■■■■■■■      ■■■■■■■■      ■■■■■■■■      ■■■■■■■■
■      ■      ■      ■      ■      ■      ■
■■searchFlight>■      ■      ■      ■      ■
■      ■■■searchAvail()>■      ■      ■
■      ■<■■flightList■■      ■      ■
■<■showResults■      ■      ■      ■
■      ■      ■      ■      ■      ■
■■selectFlight>■      ■      ■      ■      ■
■■enterDetails>■      ■      ■      ■      ■
■      ■■■bookSeat()■■>■      ■      ■
■      ■      ■      ■      ■      ■
■      ■      ■■■processPayment()■■■■■■■■■■>■ [async]
■      ■<■■bookingRef■■      ■      ■
■<■showBooking■      ■      ■      ■
■      ■      ■      ■■■■sendEmail()>■
■      ■      ■      ■      ■      ■
■<■■■■■■■■■■■■■■■■■■■■[async email notification]■■■■■■■■

```

SECTION 4 — StarUML PRACTICAL GUIDE

4.1 Creating a New Project

Step	Action	Where to Click
1	Open StarUML	Launch from desktop/start menu
2	File → New	Top menu bar → File → New
3	Model Explorer appears	Left side panel shows 'Model'
4	Right-click 'Model' → Add Diagram	Choose diagram type from popup
5	Select 'Use Case Diagram' or other	List of UML diagram types appears
6	Diagram canvas opens	White drawing area in centre

4.2 Use Case Diagram in StarUML

Element	How to Add	Tip
Actor	Toolbox → Actor → Click on canvas	Name appears below stick figure
Use Case	Toolbox → UseCase → Click/drag on canvas	Rename by double-clicking
System Boundary	Toolbox → System → Draw rectangle	Drag to resize boundary
Association	Click Actor → drag to Use Case	Line auto-appears
<<include>>	Toolbox → Include → drag from Base to Included	Add <<include>> label
<<extend>>	Toolbox → Extend → drag from Extension to Base	Add <<extend>> label
Generalization	Toolbox → Generalization → Child to Parent	Triangle at parent end

4.3 Class Diagram in StarUML

Action	Steps in StarUML
Add a Class	Toolbox → Class → Click canvas → Name it
Add Attribute	Double-click class → Attributes tab → + button → Enter name:type
Add Method	Double-click class → Operations tab → + button → Enter name():return
Set Visibility	Click attribute/method → Properties panel → Visibility dropdown
Add Association	Toolbox → Association → drag from Class1 to Class2
Add Inheritance	Toolbox → Generalization → drag from Child to Parent
Add Composition	Toolbox → Composition → drag from Whole to Part
Add Aggregation	Toolbox → Aggregation → drag from Whole to Part

Action	Steps in StarUML
Add Interface	Toolbox → Interface → click canvas → Add methods
Add Realization	Toolbox → Realization → drag from Class to Interface
Set Multiplicity	Click on relationship line → Properties → set multiplicity

4.4 Sequence Diagram in StarUML

Action	Steps
Add Actor/Object	Toolbox → Actor or Lifeline → Click canvas → Name it
Add Message	Toolbox → Message → Click source lifeline → drag to target
Add Return Message	Toolbox → Return → drag back from target to source
Add Self-call	Toolbox → Message → drag from lifeline back to SAME lifeline
Add Loop/Alt frame	Toolbox → CombinedFragment → select type (loop/alt/opt)
Reorder messages	Drag messages up/down the lifeline to change sequence
Label messages	Double-click message arrow → type method name

4.5 Exporting Diagrams

To export your diagram as an image for submission:

1. Select the diagram in Model Explorer
2. File → Export Diagram As Image
3. Choose format: PNG (best for submission) or SVG
4. Select save location → Click Export
5. For PDF: File → Print → Select 'Save as PDF' printer

Common StarUML Mistakes

- Drawing associations between two Use Cases (wrong — use include/extend)
- Placing Actor INSIDE the system boundary rectangle
- Arrow direction wrong for include/extend (most common exam mistake)
- Forgetting to add multiplicity on class diagram relationships
- Using filled diamond for aggregation (should be hollow)
- Not labeling the <<include>> or <<extend>> on the dashed arrow
- Missing method return types in class diagram

SECTION 5 — EXAM & VIVA PREPARATION

5.1 How to Identify Which Diagram to Draw

If question says...	Draw This Diagram
'Identify actors and use cases'	Use Case Diagram
'Show who interacts with system'	Use Case Diagram
'Functional requirements'	Use Case Diagram
'Classes, attributes, methods'	Class Diagram
'Static structure of system'	Class Diagram
'Relationships between objects'	Class Diagram
'Message flow', 'sequence of events'	Sequence Diagram
'How objects interact over time'	Sequence Diagram
'Dynamic behaviour for a scenario'	Sequence Diagram

5.2 Viva Questions — Use Case Diagram

Q: What is a Use Case Diagram?

A: It shows actors (users/systems) and use cases (functions) of a system, and their relationships. Captures functional requirements visually.

Q: What is an Actor?

A: An entity outside the system that interacts with it. Can be a person (Customer, Admin) or external system (Bank, PaymentGateway).

Q: What is the difference between include and extend?

A: <> is mandatory — always happens. <> is optional — happens only under certain conditions.

Q: Which direction does the include arrow point?

A: FROM base use case TO the included use case. Arrow → points to the use case being included (called).

Q: Which direction does the extend arrow point?

A: FROM the extension TO the base use case. Arrow → points to the base case being extended.

Q: What is Generalization in Use Case?

A: When one actor or use case is a specialization of another. Child inherits all properties of parent. Shown with hollow triangle arrow pointing to parent.

Q: What is a System Boundary?

A: A rectangle that defines the scope of the system. Actors are outside; Use Cases are inside.

Q: Can an actor be a system?

A: Yes. External systems that interact with your system are secondary actors (e.g., Payment Gateway, Email Server).

Q: What is the difference between Primary and Secondary actor?

A: Primary actor initiates the use case (e.g., Customer). Secondary actor supports/reacts (e.g., Bank System).

Q: Can two use cases have an association?

A: No. Association is only between Actor and Use Case. Between use cases, use <>, <>, or generalization.

5.3 Viva Questions — Class Diagram

Q: What is a Class Diagram?

A: Shows the static structure of a system — classes, attributes, methods, and their relationships.

Q: What is the difference between aggregation and composition?

A: Aggregation: weak 'has-a' — parts can exist without whole (hollow diamond ◻). Composition: strong 'part-of' — parts cannot exist without whole (filled diamond ◆).

Q: What does + - # mean?

A: + = public (anyone), - = private (only class itself), # = protected (class + subclasses).

Q: What is multiplicity?

A: Numbers on a relationship line showing how many objects of each class can participate. E.g., 1..* means one to many.

Q: What is the difference between inheritance and realization?

A: Inheritance: solid line + hollow triangle, class extends another class. Realization: dashed line + hollow triangle, class implements an interface.

Q: What is an interface?

A: A contract with method signatures but no implementation. Classes that implement it must provide all methods. Shown with <> stereotype.

Q: What is dependency?

A: Weakest relationship — dashed arrow. Class A temporarily uses Class B (as method parameter or local variable), but doesn't store it.

Q: When to use composition vs aggregation?

A: Composition: when part cannot logically exist alone (Order → OrderItem). Aggregation: when part can exist independently (Library → Book).

5.4 Viva Questions — Sequence Diagram

Q: What is a Sequence Diagram?

A: Shows how objects interact in time order — the sequence of messages exchanged to complete a scenario.

Q: What is a lifeline?

A: A dashed vertical line below each object, representing its existence over the duration of the interaction.

Q: What is an activation bar?

A: A thin rectangle on a lifeline showing that the object is currently executing (active/processing).

Q: What is a synchronous message?

A: A solid arrow where the sender waits for the receiver to finish before continuing. Most common type.

Q: What is an asynchronous message?

A: Sender does NOT wait — it fires the message and continues. Used for events, emails, notifications.

Q: What is a return message?

A: A dashed arrow going back from receiver to sender, showing the return value of a method call.

Q: How is time shown in sequence diagram?

A: Time flows DOWNWARD. Messages higher up happen first; messages lower happen later.

Q: What is the alt fragment?

A: Combined fragment labeled 'alt' with a dividing line — represents IF-ELSE conditional logic.

Q: What is the loop fragment?

A: Combined fragment labeled 'loop' — represents a repeated sequence of messages (like a for/while loop).

5.5

MASTER CHEAT SHEET — Quick Revision

■ ASCII DIAGRAM

USE CASE DIAGRAM CHEAT SHEET

Actor = Stick figure (OUTSIDE boundary)
 Use Case = Oval (INSIDE boundary)
 Association = Solid line (Actor ■■ UseCase)
 <<include>> = Dashed arrow → MANDATORY (Base→Included)
 <<extend>> = Dashed arrow → OPTIONAL (Extension→Base)
 Generaliz. = Solid + hollow triangle (Child→Parent)

INCLUDE keywords: always, must, every time, mandatory
 EXTEND keywords: sometimes, if, optional, conditional

CLASS DIAGRAM CHEAT SHEET

+ public - private # protected ~ package
 ■■■■■■ = Association (knows each other)
 - - - -> = Dependency (uses temporarily)
 ■■■■■■ = Aggregation (has-a, weak, hollow diamond)
 ◆■■■■■ = Composition (part-of, strong, filled diamond)
 ■■■■ = Inheritance (is-a, solid+hollow triangle)
 - - - ■ = Realization (implements, dashed+hollow tri)
 Multiplicity: 1 * 0..1 1..* 0..*

SEQUENCE DIAGRAM CHEAT SHEET

Objects = Boxes at TOP
 Lifelines = Dashed vertical lines going DOWN
 Activation = Thin rectangle ON lifeline (when active)
 Message = Solid arrow → (synchronous, waits)
 Return = Dashed arrow ← (return value)
 Async = Half-arrowhead (fire & forget)
 Self-call = Arrow loops back to same object
 Time flows DOWNWARD ↓
 loop[] alt[] opt[] = Combined fragments

5.6 Common Exam Mistakes & How to Avoid Them

Mistake	Correct Approach
Drawing arrow from Included → Base for include	Arrow goes Base → Included (<<include>>)
Drawing arrow from Base → Extension for extend	Arrow goes Extension → Base (<<extend>>)
Placing actor INSIDE system boundary	Actors are ALWAYS outside the rectangle
Using association between two use cases	Use <<include>>, <<extend>>, or generalization
Using hollow diamond for composition	Composition = FILLED diamond ◆
Using filled diamond for aggregation	Aggregation = HOLLOW diamond ■

Mistake	Correct Approach
Arrow points to CHILD in inheritance	Arrow points to PARENT (hollow triangle)
Forgetting multiplicity labels	Always add numbers (1, *, 0..1, 1..*) on lines
Forgetting return arrows in sequence diagram	Every method call should have a return (dashed)
No activation bars in sequence diagram	Show activation bar whenever object is processing
Writing attributes without visibility symbol	Always prefix with +, -, or #
Forgetting <<interface>> label on interface	Mark interfaces with <<interface>> stereotype

5.7 Memory Tricks & Mnemonics

■ ASCII DIAGRAM

INCLUDE vs EXTEND:

'I INCLUDE you ALWAYS' = mandatory, common sub-step

'I EXTEND only SOMETIMES' = optional, conditional

ARROW DIRECTION TRICK:

Think of arrow as 'pointing to what is NEEDED/USED'

include: Base needs Included → arrow → to Included

extend: Extension needs Base to exist → arrow → to Base

DIAMONDS (Class Diagram):

Hollow ■ = Aggregation = 'A-ggregation, A-ir (not solid)'

Filled ◆ = Composition = 'C-omposition, C-oncrete (solid)'

RELATIONSHIPS STRENGTH (weakest to strongest):

Dependency → Association → Aggregation → Composition

'DAAC' = 'Developers Always Add Code'

VISIBILITY:

+ = Plus = Public = Everyone can use

- = Minus = Private = Minus others = only me

= Hash = Protected = Hashtag family (subclasses)

SEQUENCE DIAGRAM:

'Time goes Down like water falling'

'Activation bar = Object is ACTIVE (working)'

'Return is DASHED because it's the response, not request'

GOOD LUCK IN YOUR EXAM! ■

You have everything you need:

- All 3 UML diagram types explained from basics to advanced
- Every symbol, arrow, and relationship with ASCII visuals
- 5 complete example diagrams for each type

- StarUML step-by-step guide
- Viva questions with detailed answers
- Common mistakes to avoid
- Memory tricks and cheat sheet

Last Minute Exam Tips

- Read the question CAREFULLY — identify the system and actors first
- For Use Case: list all actors, then list all functions, then add relationships
- For Class: identify nouns as classes, verbs as methods, adjectives as attributes
- For Sequence: trace the scenario step by step, identify who sends each message
- Always label your <<include>> and <<extend>> arrows clearly
- Double-check arrow directions before submitting
- Add multiplicity to all class diagram relationships
- Show activation bars on sequence diagram lifelines